

lib\pages\home_page.dart

```
1 import 'package:flutter/material.dart';
2 // import libreria http + alias
3 import 'package:http/http.dart' as http;
4 // import dart convert
5 import 'dart:convert';
6 // import story_model.dart
7 import 'package:hacker_news_api/models/story_model.dart';
8
9 /// classe HomePage
10 class HomePage extends StatefulWidget {
11   const HomePage({super.key});
12
13   // override del metodo createState
14   @override
15   State<HomePage> createState() => _HomePageState();
16 }
17
18 /// classe HomePageState
19 class _HomePageState extends State<HomePage> {
20   /// variabile di stato storyModelFuture che conterrà
21   /// un qualcosa che verrà risolto in futuro e quel
22   /// qualcosa è un'istanza di StoryModel
23   late Future<List<StoryModel>> storyModelFuture;
24
25   /// numero di topStories da prendere
26   final int firstTopStories = 15;
27
28   // override del metodo initState
29   @override
30   void initState() {
31     // super initState
32     super.initState();
33
34     /// inizializzazione di storyModelFuture
35     storyModelFuture = downloadStoryData();
36   }
37
38   /// metodo downloadStoryData
39   Future<List<StoryModel>> downloadStoryData() async {
40     /// chiamata http per prendere le topstories
41     final topStoriesHttpResponse = await http.get(
42       Uri.parse(
43         "https://hacker-news.firebaseio.com/v0/topstories.json?print=pretty",
44       ),
45     );
46
47     /// parsing di topStoriesHttpResponse
48     /// che restituisce una lista di interi (id)
49     final topStoriesIds =
50       json.decode(topStoriesHttpResponse.body) as List<dynamic>;
51
```

```

52     /// ciascuno di questi id viene mappato
53     /// (si prendono solamente le prime firstTopStories)
54     final topStoriesFutures = topStoriesIds.take(firstTopStories).map((
55         storyId,
56     ) async {
57         /// si deve iterare/ciclare per ciascun id/elemento
58         /// e per ciascun id fare un'altra chiamata http
59
60         /// altra chiamata http per ciascun id, http.get
61         final response = await http.get(
62             Uri.parse(
63                 "https://hacker-news.firebaseio.com/v0/item/$storyId.json?print=pretty",
64             ),
65         );
66
67         /// storyData: risultato di json.decode
68         /// e parsing del json
69         final storyData = json.decode(response.body);
70
71         /// ritorna un'istanza di StoryModel
72         /// invocando il metodo fromData
73         return StoryModel.fromData(storyData);
74     }).toList();
75
76     /// aspettare che le topStoriesFutures si completino,
77     /// ovvero si aspetta che tutte le chiamate http che
78     /// avvengono in parallelo (e che prendono i dettagli
79     /// delle stories) vengano completate
80     final topStories = await Future.wait(topStoriesFutures);
81
82     // valore di ritorno
83     return topStories;
84 }
85
86 /// metodo refreshData per aggiornare la lista delle stories
87 void refreshData() {
88     // setState
89     setState(() {
90         ///
91         storyModelFuture = downloadStoryData();
92     });
93 }
94
95 // override del metodo build
96 @override
97 Widget build(BuildContext context) {
98     /// Scaffold
99     return Scaffold(
100         /// appBar
101         appBar: AppBar(
102             /// title di AppBar con Container
103             title: Container(
104                 /// padding
105                 padding: EdgeInsets.symmetric(horizontal: 10, vertical: 8),

```

```

106         decoration: BoxDecoration(
107             color: Colors.black,
108             borderRadius: BorderRadius.circular(10),
109         ),
110         child: Text(
111             "News!",
112             // style di Text
113             style: TextStyle(
114                 // colore
115                 color: Colors.white,
116                 // dimensioni
117                 fontSize: 17,
118                 // grassetto
119                 fontWeight: FontWeight.bold,
120             ),
121         ),
122     ),
123
124     /// actions
125     actions: [
126         IconButton(
127             // onPressed per il refresh
128             onPressed: refreshData,
129             // icona di refresh
130             icon: Icon(Icons.refresh_rounded),
131         ),
132     ],
133
134     /// Title al centro (AppBar)
135     centerTitle: true,
136
137     /// colore di sfondo (AppBar)
138     backgroundColor: Colors.white,
139 ),
140
141 /// body
142 body: Center(
143     /// child
144     child: body(),
145 ),
146 );
147 }
148
149 /// body: componente personalizzato
150 Widget body() => FutureBuilder<List<StoryModel>>(
151     /// future su cui ci si mette in ascolto
152     future: storyModelFuture,
153     // builder
154     builder: (context, snapshot) {
155         /// se questa future non è stata ancora risolta
156         if (snapshot.connectionState != ConnectionState.done) {
157             // caricamento/loading
158             return Container(
159                 // colore di sfondo (Container)

```

```

160         color: Colors.white,
161         // allineamento del Container
162         alignment: Alignment.center,
163         // child di Container
164         child: CircularProgressIndicator(
165             // colore della CircularProgressIndicator
166             color: Colors.black,
167         ),
168     );
169 } else {
170     // ritorna un Container
171     return Container(
172         /// colore Container
173         color: Colors.white,
174
175         /// child: ListView (variante separated)
176         child: ListView.separated(
177             /// itemCount
178             itemCount: snapshot.data!.length,
179
180             /// itemBuilder
181             itemBuilder: (context, index) => ListTile(
182                 /// title (titolo)
183                 title: Text(
184                     snapshot.data![index].title,
185                     style: TextStyle(fontWeight: FontWeight.bold),
186                 ),
187
188                 /// subtitle (autore)
189                 subtitle: Text(
190                     // autore
191                     snapshot.data![index].author,
192                     // stile di Text per autore
193                     style: TextStyle(fontStyle: FontStyle.italic),
194                 ),
195             ),
196
197             /// separatorBuilder
198             separatorBuilder: (context, index) => Divider(color: Colors.white),
199         ),
200     );
201 }
202 },
203 );
204 }
205

```